

Assignment 3: Arrays and the Simplest Useful ADT

Auqib Hamid Lone

Department of Computer Science & Engineering, GCET Ganderbal

September 9, 2026

Due: two weeks from issue. There is no automatic late penalty — if you need more time, ask before the deadline and an extension will be granted (see the course policy in the syllabus).

Answer every question. Show all working for Numerical and Design questions. Each problem states its own assumptions; everything you need is in Week 3’s Notes chapter alone. Notation matches the lecture exactly: n counts elements, **base** is the address of $A[0]$, w is bytes per element, **top** = -1 means empty, **MAX** is the array capacity.

3.1 Conceptual

Q1. An operand table holds n values in an array. Accessing $A[i]$ costs $O(1)$ worst case whatever i is, yet inserting a new value at position 0 costs $O(n)$ worst case.

- (a) In one or two sentences, explain why the *position* i never affects access cost (name the mechanism that locates $A[i]$).
- (b) In one or two sentences, explain why the *position* of an insert decides its cost, and why position 0 is the most expensive case (state how many elements move when $n = 1000$).
- (c) A classmate claims “the address formula makes room for the new element, so nothing needs to shift.” In one sentence, state the confusion behind this claim (locating a cell vs. freeing a cell).

Why this matters: the calculator stores operands in order and wants the i -th one instantly — this question pins down exactly what arrays promise (instant reads) and what they do not (cheap splices).

Q2. The stack contract names three operations — **push(x)**, **pop()**, **peek()** — with last-in-first-out discipline (one of Sedgewick’s three collection types, p.120) [2]; the lecture implements them over one array and one index **top** (Karumanchi Ch. 4) [1].

- (a) State which of the three operations remove an item, which only inspect it, and what each one reports or returns on an empty stack in the lecture’s implementation (name **underflow**).
- (b) State the exact condition under which **push** must refuse in this implementation, and name the error (**overflow**).
- (c) In one or two sentences, explain why fixed capacity is the price of this implementation — what must happen when the array fills, and which later structure (Week 6) removes the ceiling.

Why this matters: naming the contract before coding it is the shape every week repeats — this question checks you can tell the promise (LIFO) from one particular way of keeping it (an array plus **top**).

3.2 Numerical

Q3. A program declares `int A[12]` with `base = 3000` and $w = 4$, and separately `int B[5][6]` with `base = 4000` and $w = 4$. All indices are 0-based and every access below is in range.

- (a) Compute $\text{addr}(A[7])$ using $\text{addr}(A[i]) = \text{base} + i \times w$. Show the multiply and the add as separate steps.
- (b) Compute the *row-major* address of `B[3][4]` using $\text{addr}(B[i][j]) = \text{base} + ((i \times c) + j) \times w$ with $c = 6$. Show the flat index $k = (i \times c) + j$ and the byte offset as separate steps.
- (c) Compute the *column-major* address of `B[3][4]` on the same array using $\text{addr}(B[i][j]) = \text{base} + ((j \times r) + i) \times w$ with $r = 5$. Show $k = (j \times r) + i$ and the byte offset as separate steps, then state in one sentence why (b) and (c) differ (row-major skips rows; column-major skips columns).

Why this matters: C lays rows out first while Fortran/MATLAB lay columns out first — mixing the two up places the same indices in different bytes, a classic porting bug.

Q4 carries over to the next page.

Q4. Evaluate the postfix string `9 5 2 + - 4 *` with the lecture's one-stack algorithm (numbers are pushed; on an operator `op`, `b = pop()`, `a = pop()`, then push `apply(op, a, b)`):

- (a) Give the stack contents after *every* token (six tokens: `9`, `5`, `2`, `+`, `-`, `*`), presented as a small table with one row per token.
- (b) State the final answer (the last `pop()`).
- (c) In one sentence, explain why the pop order (`b` first, `a` second) matters for the `-` step in this string — what would go wrong if the two popped values were combined in the other order?

Why this matters: postfix needs no parentheses and no precedence rules — one left-to-right pass with a stack remembers the pending operands, which is exactly what the expression processor exploits.

3.3 Design

Q5. Run the lecture's bracket-checking algorithm (openers are pushed; on a closer, report unbalanced if the stack is empty or the top is not its match, else pop; at the end, balanced iff the stack is empty) on each string below. For each string, give a token-by-token trace (token, stack after the step) and the verdict.

- (a) `{[( ) ( )]}`
- (b) `([{ ]})`

For the unbalanced string, name the first step that disagrees (the token, the stack contents at that step, and why it is a mismatch). For the balanced string, confirm the stack is empty at the end. Why this matters: the last-opened bracket must close first — exactly LIFO — so one stack decides nesting correctness in a single pass at $O(n)$ time.

Q6. A stack is implemented exactly as in the lecture with `MAX = 4` and `top = -1` (empty). Starting from empty, execute this sequence:

`push(11), push(22), pop(), push(33), push(44), push(55), push(66)`

- (a) Trace `top` after every step (the index path), state what `pop()` returns, and state which `push` — if any — reports overflow (`top + 1 == MAX`).
- (b) State the final `top` and the value `peek()` returns.
- (c) In one sentence, state the cost of each of `push`, `pop`, `peek` in this implementation (bound plus case, in Week 2's vocabulary) and the one array access plus one index update that buys it.

Why this matters: `top` is the whole discipline — tracing the index rather than just the values is what lets you predict overflow, underflow, and the final state before running anything.

References

- [1] Narasimha Karumanchi. *Data Structures And Algorithms Made Easy*. CareerMonk Publications, 2017. This calibre-generated edition carries no printed folio numbers; page numbers cited in CS301 material are PDF page indices. NOT a source for Week 1's C-specific pointer/malloc content — see this course's supporting-books index for the coverage check.
- [2] Robert Sedgewick and Kevin Wayne. *Algorithms*. Addison-Wesley, 4th edition, 2011.